Luồng hệ thống IELTS Reading và kiến thức Backend cho phỏng vấn

Bản đối chiếu với mã nguồn hiện tại, ngày 11 tháng 11 năm 2026. Tài liệu này giúp bạn kể lại một yêu cầu đi qua React, Spring Boot và PostgreSQL, giải thích vì sao code được tổ chức như hiện tại, rồi nêu giới hạn một cách trung thực khi phỏng vấn Backend Intern hoặc Spring Developer. Phạm vi chỉ gồm IELTS Academic Reading.

Cách đọc: đi theo mục 1 đến 8 để nắm luồng thực thi; dùng mục 9 đến 12 để ôn OOP, giao dịch, bảo mật, kiểm thử và tập trả lời. Các đường dẫn trong tài liệu tính từ thư mục gốc dự án. Đây là bản phân tích code, không phải biên bản xác nhận mọi chức năng đã được kiểm thử thành công trên môi trường chạy thực tế.

Điểm phỏng vấn: Dự án dùng Java 21, Spring Boot 4.1.1, Spring Security session, JdbcTemplate, Flyway, PostgreSQL, React và TypeScript. Không mô tả nó như ứng dụng JPA hoặc JWT vì hai kỹ thuật đó chưa được triển khai.

1 Bức tranh kiến trúc và đường đi của một yêu cầu

Frontend dùng React 19, TypeScript, React Router và Vite. Trong môi trường phát triển, Vite chuyển tiếp `/api`, `/auth` và `/me` đến backend. Trình duyệt gọi `fetch` qua `frontend/src/api.ts`, gửi cookie cùng nguồn bằng `credentials: same-origin`; các lệnh ghi dữ liệu lấy CSRF token trước. Backend dùng Spring MVC nhận HTTP, Spring Security xác thực và phân quyền, JdbcTemplate chạy SQL có tham số, PostgreSQL lưu dữ liệu. Flyway áp dụng migration khi ứng dụng khởi động. Docker Compose ghép PostgreSQL, API và dịch vụ OCR ở môi trường local.

```
React component → api.ts → SecurityFilterChain → Controller hoặc Service
                → JdbcTemplate → PostgreSQL → DTO record → JSON → React
```

`identity` và `content` là hai gói nghiệp vụ chính trong backend. Một số luồng có Service riêng như `DictionaryLookupService`, `AcademicBandService` và các PDF service; nhiều content controller vẫn chứa SQL và quy tắc nghiệp vụ. Đây là cấu trúc thực tế của MVP, chưa phải mô hình Controller–Service–Repository tách hoàn toàn. Cách hiện tại giúp đi nhanh và nhìn rõ truy vấn, nhưng controller lớn gây khó kiểm thử độc lập, tái sử dụng và bảo trì.

Thành phần	Vai trò thực tế	Nơi đối chiếu
Frontend	Form, admin biên soạn, màn làm bài hai cột, kết quả, từ vựng, dashboard	frontend/src/App.tsx, frontend/src/features
Identity	Đăng ký, đăng nhập session, hồ sơ, đổi mật khẩu, quyền ADMIN	backend/.../identity/AuthController.java, SecurityConfiguration.java
Content	Passage, câu hỏi, full test, attempt, chấm điểm, từ điển, PDF	backend/.../content/*.java
Database	Quan hệ, ràng buộc, chỉ mục, lịch sử, band, từ điển	backend/src/main/resources/db/migration/ V1…V25
Triển khai local	API port 8080, PostgreSQL host port 5433, OCR service	compose.yaml, frontend/vite.config.ts

Một yêu cầu ví dụ: user bấm lưu đáp án. React gửi PUT và CSRF token. Spring Security khôi phục Authentication từ session, Controller lấy user ID từ Principal, xác minh attempt và question, JdbcTemplate

UPDATE hoặc INSERT đáp án trong transaction. Database giữ UNIQUE theo cặp attempt–question. Backend trả HTTP; React cập nhật trạng thái. Mỗi tầng có trách nhiệm riêng: UI phản hồi sớm, backend giữ luật nghiệp vụ và quyền, database giữ tính toàn vẹn khi request cạnh tranh.

2 Identity từ đăng ký đến đăng xuất

2.1 Đăng ký

1. React kiểm tra email, tên, xác nhận mật khẩu và độ dài UTF-8 để báo lỗi sớm. Đây là hỗ trợ UX; client có thể bị bỏ qua nên backend tự kiểm tra lại.
2. `POST /auth/register` nhận `Registration record`. Backend chuẩn hóa email bằng `trim` và `toLowerCase(Locale.ROOT)`, kiểm tra dạng email, tên 1–100 ký tự, mật khẩu tối thiểu 8 ký tự và tối đa 72 byte UTF-8.
3. `PasswordEncoder` dùng BCrypt tạo hash trước khi INSERT. SQL cố định role là `LEARNER`, client không thể tự đăng ký `ADMIN`. PostgreSQL có `UNIQUE(email_normalized)` để chặn email trùng ngay cả khi hai request đồng thời; `DuplicateKeyException` đổi thành HTTP 409.

```
record Credentials(String email, String password) {}
UNIQUE (email_normalized)  →  409 khi email đã có
```

`Credentials` là DTO request bất biến ở mức tham chiếu, không phải bản ghi session hay bảng users. Record tự sinh constructor, accessor, `equals`, `hashCode`, `toString`; vì `toString` có thể chứa password, tránh log nguyên DTO. BCrypt có salt và work factor nên cùng mật khẩu có thể cho hash khác nhau; xác thực bằng `matches(raw, hash)`, không hash lại rồi so chuỗi. Giới hạn 72 byte tránh BCrypt bỏ qua phần sau giới hạn; byte khác số ký tự, đặc biệt với Unicode.

Điểm phỏng vấn: Kiểm tra ở frontend, kiểm tra nghiệp vụ ở backend và ràng buộc UNIQUE của database giải quyết ba vấn đề khác nhau. Frontend không thay backend, backend không thay cơ chế chống race của database.

2.2 Đăng nhập và session

1. `POST /auth/sessions` tìm user bằng `email_normalized`. Nếu không có tài khoản, code vẫn gọi BCrypt với dummy hash để chi phí xử lý gần nhau. Email sai, mật khẩu sai và tài khoản bị khóa cùng trả 401 với thông báo chung.
2. Khi đúng, backend đổi session ID nếu đã có session, tạo `UsernamePasswordAuthenticationToken` với principal là user ID dạng chuỗi và quyền `ROLE_LEARNER` hoặc `ROLE_ADMIN`, rồi lưu `SecurityContext` qua `HttpSessionSecurityContextRepository`.
3. Browser giữ cookie session. Spring Security khôi phục Authentication ở request kế tiếp. `GET /me` tra lại bảng users theo ID để lấy email, tên và role hiện thời; React gọi endpoint này khi mở lại trang để biết phiên đăng nhập còn hợp lệ không.

Session là trạng thái đăng nhập phía server; `Credentials` chỉ mang email/mật khẩu trong request. Cookie chứa mã định danh để tham chiếu session, không chứa password. `request.changeSessionId()` giảm rủi ro session fixation. Cookie cấu hình HttpOnly, SameSite=Lax; cờ Secure lấy từ `SESSION_COOKIE_SECURE` và cần bật khi chạy HTTPS.

`SecurityFilterChain` cho phép đăng ký, đăng nhập, lấy CSRF token, health và danh sách Reading công khai; `/api/admin/**` yêu cầu `ROLE_ADMIN`; những request khác cần đăng nhập. Guard ở React là trải nghiệm điều hướng, không thay kiểm tra server. CSRF vẫn bật: `api.ts` gọi `GET /auth/csrf` trước POST, PUT, PATCH, DELETE rồi gửi token trong header.

```
/auth/csrf → token → POST /auth/sessions → cookie session
GET /me + cookie → Authentication(userId, ROLE_*) → hồ sơ user
```

Điểm phỏng vấn: Ứng dụng web cùng nguồn phù hợp session; Spring Security hỗ trợ vòng đời session và CSRF. JWT là lựa chọn có đánh đổi riêng về expiry, refresh và revoke nhưng chưa dùng ở dự án.

2.3 Hồ sơ, đổi mật khẩu, đăng xuất

`PATCH /me` chỉ cho sửa `display_name`; email và role không được client cập nhật. `PATCH /me/password` kiểm mật khẩu cũ bằng BCrypt, buộc mật khẩu mới khác cũ, sau đó UPDATE có điều kiện `user_id AND password_hash AND is_active`. Nếu request khác đã thay hash trước đó thì số dòng cập nhật bằng 0 và API trả 409. Đây là compare-and-swap ở SQL, một ví dụ optimistic concurrency nhỏ, không phải `@Version` của JPA. `DELETE /auth/sessions/current` xóa SecurityContext và invalidate session.

```
UPDATE users SET password_hash = ?
WHERE user_id = ? AND password_hash = ? AND is_active = true
```

Giới hạn còn lại: chưa có xác minh email, quên mật khẩu, rate limiting đăng nhập, session phân tán hoặc thu hồi mọi session khi đổi mật khẩu. Đây là các việc cần đánh giá trước public launch. Nguồn: `identity/AuthController.java`, `identity/SecurityConfiguration.java`, `application.yaml`, `frontend/src/api.ts`, migration V1.

3 PostgreSQL và Flyway quản lý dữ liệu ra sao

Flyway đọc V1 đến V25 theo thứ tự và ghi trạng thái vào `flyway_schema_history`. Mỗi migration thay đổi cấu trúc hoặc chuyển dữ liệu. Không sửa migration đã chạy trên môi trường có dữ liệu; hãy tạo version mới. PostgreSQL dùng BIGINT IDENTITY cho PK, FK cho quan hệ, CHECK cho trạng thái và miền giá trị, UNIQUE cho tính duy nhất, `timestamp with time zone` cho thời điểm. API dùng `?` của JdbcTemplate để bind tham số, tránh ghép trực tiếp input vào SQL.

Miền dữ liệu	Bảng then chốt	Quan hệ cần giải thích
Tài khoản	users	Một user có nhiều attempt, category và note
Nội dung	passages, question_groups, passage_questions	Một passage có nhiều nhóm; một nhóm có nhiều câu
Đáp án	option_questions, question_answer_options, blank_questions, blank_answer_variants, matching_questions, matching_question_answers	Ba họ OPTION, BLANK, MATCHING có cấu trúc đáp án riêng
Full test	reading_tests, test_passages	Một test gồm ba passage, lưu thứ tự và khoảng số câu
Lượt làm	quiz_attempts, user_answers, reading_test_attempts, reading_test_answers	Đáp án luôn gắn một attempt cụ thể
Từ vựng	vocabulary_entries, vocabulary_senses, meanings, phonetics, notes, categories, note_categories	Từ điển dùng chung; ghi chú/danh mục thuộc user
Phụ trợ	reading_highlights, academic_band_tables, academic_band_ranges, pdf_import_jobs	Highlight theo attempt; band versioned; PDF job và review

Ràng buộc nhiều dòng như đúng ba passage, tổng 40 câu, số liên tục 1…40 không thể biểu diễn đầy đủ bằng CHECK của một hàng. `AdminReadingTestController.composition` kiểm tra trước khi lưu và lặp lại khi

publish. Database vẫn giữ CHECK/UNIQUE/FK cho bất biến ở mức hàng và quan hệ. `test_passages` lưu khoảng số câu toàn đề; passage tự đánh số cục bộ 1…n. Do đó số câu có thể là 13/13/14 hoặc 14/13/13, miễn tổng 40 và liên tục.

JdbcTemplate làm SQL và JOIN hiển lộ, giúp học cách đọc query và chủ động chọn cột. Đổi lại code tự mapping ResultSet, tự bảo trì truy vấn phức tạp và dễ lặp SQL. `@Transactional` đặt ranh giới giao dịch cho các câu lệnh phải cùng thành công hoặc cùng thất bại; `FOR UPDATE` khóa hàng khi hai request cùng đổi trạng thái; UNIQUE là tuyến cuối để giữ dữ liệu không trùng.

Điểm phỏng vấn: Phân biệt pessimistic lock `SELECT … FOR UPDATE`, optimistic compare-and-swap `WHERE version = ?` và UNIQUE. Chúng xử lý các vấn đề cạnh tranh khác nhau và có thể cùng tồn tại.

4 Luồng quản trị nội dung và xuất bản

4.1 Tạo passage và nhóm câu hỏi

1. Admin gọi `/api/admin/passages` tạo passage DRAFT với title, body và partNumber 1–3. Bản nháp được sửa; sau PUBLISHED, API không cho sửa trực tiếp để tránh đổi nội dung khi đã có người làm.
2. Admin tạo question group với `taskVariant`, instructions, questions và đáp án. `AdminQuestionGroupController` kiểm số câu, dạng, key, lựa chọn, giới hạn từ và template trước khi ghi. Tạo/sửa/xóa nhóm chạy trong transaction và chỉ trên passage DRAFT.
3. Khi thêm hoặc xóa câu, `renumber` sắp theo `display_order` rồi đánh lại 1…n. Code tạm chuyển số cũ lên vùng cao để tránh va UNIQUE trong lúc UPDATE. ID câu giữ ổn định dù số hiển thị đổi.

Ba họ dữ liệu: OPTION giữ lựa chọn và cờ đúng cho True/False/Not Given, Yes/No/Not Given và multiple choice. BLANK giữ đáp án chính, biến thể hợp lệ, word limit cho sentence, note, summary, table, flow chart và short answer. MATCHING giữ pool candidate, nhãn và liên kết đáp án giữa câu với candidate; `candidate_reuse` thể hiện khả năng dùng lại. Một ô trống trong đoạn văn vẫn là một `passage_question` có ID và số riêng.

`presentation_template` lưu bố cục đoạn, bảng hoặc flow để đặt từng ô. `PresentationTemplateCodec` dùng Jackson ghi/đọc, và khi trả cho learner sẽ bỏ `sourceText` nội bộ. Template phục vụ trình bày, còn key chuẩn nằm trong bảng đáp án. Phân tách này tránh lộ đáp án chỉ vì frontend cần dựng giao diện.

Endpoint `publish-check` trả danh sách lỗi. Khi publish, backend khóa passage bằng `FOR UPDATE`, kiểm lại title/body, nhóm, câu, đáp án và tính liên tục rồi mới đổi status. Admin có thể publish passage lẻ; full test là bước ghép riêng.

Điểm phỏng vấn: Phải kiểm lại ở lúc publish vì bản nháp có thể đổi giữa preview và nút publish. Validation ở UI không phải bằng chứng cuối cùng.

4.2 Ghép full test 40 câu

`AdminReadingTestController` nhận ba passage đã xuất bản, không trùng ID, đúng partNumber 1, 2, 3. Mỗi passage phải có số cục bộ từ 1 đến số cuối không thiếu; tổng ba passage là 40. Code tính `question_start_number` và `question_end_number` nối tiếp rồi lưu `test_passages`. `publish-check` và `publish` cùng dùng composition; publish khóa test DRAFT. Màn làm bài dùng offset của passage cộng số cục bộ để hiện số toàn đề.

```
globalQuestionNumber = question_start_number + localQuestionNumber - 1
```

Diagram Label Completion chưa nằm trong danh sách task variant của schema vì chưa có luồng quản lý hình ảnh. Không nên nói hệ thống đã hỗ trợ đủ mọi dạng IELTS Reading. Nguồn:

AdminPassageController.java, AdminQuestionGroupController.java, AdminReadingTestController.java, PresentationTemplate.java, migration V2–V6 và V8.

5 Luồng người học làm passage lẻ

5.1 Tải dữ liệu và bắt đầu lượt làm

Người chưa đăng nhập có thể xem danh sách published passages; đọc chi tiết và làm bài yêu cầu đăng nhập. ReadingPassageController trả body, nhóm, câu, options, candidates và template; không trả is_correct, đáp án BLANK/MATCHING hay explanation trước lúc nộp. Frontend dựng hai cột bài đọc và câu hỏi, thanh số câu, thanh kéo chỉnh độ rộng trong giới hạn 30–65%, giữ phần câu hỏi có thể nhìn thấy.

POST /api/reading/passages/{passageId}/attempts bắt đầu hoặc nối tiếp attempt đang làm/tạm dừng. quiz_attempts ghi user_id, passage_id, mode PRACTICE hoặc REAL_TEST, status và elapsed_seconds. Unique active marker ngăn hai lượt đang làm cho cùng user/passage; code khóa user row để tuần tự hóa request start đồng thời. Lượt làm lại sau khi đã nộp có ID mới, nhờ vậy đáp án và highlight không lẫn với lượt cũ.

5.2 Lưu đáp án

PUT /api/reading/attempts/{attemptId}/answers/{questionId} khóa attempt, đối chiếu chủ sở hữu và câu thuộc passage, từ chối nếu không còn IN_PROGRESS. Câu điền từ giữ submitted_raw_value để hiển thị đúng như user nhập, và submitted_value chuẩn hóa bằng Unicode NFC, trim, gộp khoảng trắng, lower-case Locale.ROOT để chấm. AnswerTextPolicy đếm từ và số theo word_limit_mode; câu lựa chọn kiểm nhãn đáp án thực sự nằm trong tập lựa chọn hợp lệ. Gửi giá trị rỗng sẽ xóa đáp án đã lưu. UNIQUE(attempt, question) bảo đảm mỗi câu chỉ có một giá trị cuối cùng.

Ví dụ Vietnam, vietnam, vietNam là cùng đáp án sau chuẩn hóa chữ hoa/thường. Nếu một đáp án khác nội dung nhưng vẫn chấp nhận được, admin thêm vào blank_answer_variants; hệ thống không tự suy đoán mọi từ đồng nghĩa. Với ONE_WORD, cụm New York bị chặn vì vượt giới hạn. Quy tắc này có ở backend, không chỉ ở HTML input.

5.3 Đồng hồ và kết quả

Đồng hồ passage hiển thị theo thời gian thực ở frontend, gửi heartbeat khoảng 15 giây. Server ghi GREATEST(elapsed_seconds, số giây client báo) và cập nhật lần cuối khi pause/submit. Chuyển tab nhưng vẫn mở bài thì đồng hồ tiếp tục. Mất mạng hoặc đóng bất ngờ có thể mất đoạn thời gian chưa gửi heartbeat. Đây là thiết kế clock do frontend báo, không phải đồng hồ server tuyệt đối. Backend của passage lẻ chưa tự nộp ở mốc 60 phút cho cả PRACTICE lẫn REAL_TEST; REAL_TEST có thao tác pause/thoát và khóa tra từ riêng.

POST submit đổi status sang SUBMITTED, xóa active_marker, ghi submitted_at. Gọi lại với attempt đã nộp trả trạng thái cũ thay vì tạo lượt nộp mới. PracticeResultController chỉ trả đáp án và explanation cho đúng owner khi đã SUBMITTED; mỗi câu là CORRECT, INCORRECT hoặc SKIPPED. BLANK so đáp án chính và variants sau chuẩn hóa. Multiple choice nhiều đáp án chấm từng nhãn đúng nhưng không cộng điểm lặp cùng nhãn. Passage lẻ không đổi sang IELTS band; band chỉ tính khi full test có 40 câu.

Điểm phỏng vấn: Client làm phản hồi thời gian mượt, backend giữ dữ liệu bền vững theo mốc. Hãy nêu rõ giới hạn độ chính xác của passage timer khi mất kết nối; đừng nói server đã kiểm soát tuyệt đối nếu code chưa làm vậy. Nguồn: ReadingPassageController.java, PracticeAttemptController.java, PracticeResultController.java, AnswerTextPolicy.java, migration V7, V10, V13, V14.

6 Full test, tính giờ và band Academic Reading

Full test dùng `reading_test_attempts` và `reading_test_answers` riêng. Start/resume giữ một active attempt. Mỗi lần lưu đáp án đều xác minh câu thuộc một trong ba passage của test. Màn làm bài có số toàn đề 1…40 nhưng database vẫn lưu theo `passage_question_id`, nên dữ liệu không phụ thuộc số hiển thị.

`ReadingTestPracticeController` lưu `elapsed_seconds` cùng `clock_started_at`. Mỗi heartbeat, lưu đáp án, pause hoặc submit gọi `accrue`: lấy chênh lệch giữa `Instant.now()` và mốc clock, cộng vào tổng đã lưu, rồi đặt lại mốc. Nếu request là phục hồi sau ngắt kết nối, phần thời gian chưa checkpoint được giới hạn tối đa 15 giây. Đây là quy tắc sản phẩm để tránh cộng cả thời gian user đã đóng trình duyệt. REAL_TEST tự nộp khi đạt 3600 giây. PRACTICE tiếp tục tính giờ nhưng không tự nộp tại 60 phút.

Trong `saveAnswer`, transaction có `noRollbackFor = ResponseStatusException.class`: nếu vừa nhận câu trả lời thì nhận ra đã hết giờ, trạng thái SUBMITTED vẫn phải commit dù API trả 409 cho đáp án đến muộn. `FOR UPDATE` trên attempt ngăn heartbeat, save và submit cùng đổi đồng hồ hoặc status không nhất quán.

Khi kết thúc, code chấm 40 câu và lưu `total_answered`, `total_correct`, `academic_band_table_id`, `band_score`, `band_result_label` cùng attempt. `AcademicBandService` chọn bảng band còn hiệu lực và khoảng raw score chứa số câu đúng. Bảng trong migration V16 là hướng dẫn luyện tập, không bảo đảm y hệt mọi đề thi thật.

Số đúng	Band lưu	Số đúng	Band lưu
0–10	Below 4.0	26–29	6.5
11–12	4.0	30–31	7.0
13–15	4.5	32–34	7.5
16–17	5.0	35–36	8.0
18–22	5.5	37–38	8.5
23–25	6.0	39–40	9.0

Snapshot ở đây nghĩa là lưu band và ID phiên bản bảng quy đổi tại lúc nộp. Nếu sau này thay bảng band, kết quả cũ giữ cách diễn giải ban đầu. Hệ thống **chưa snapshot toàn bộ nội dung passage và đáp án đúng** vào attempt. Việc cấm sửa PUBLISHED hạn chế trôi dữ liệu nhưng không giống snapshot đề đầy đủ.

Điểm phỏng vấn: Full test dùng thời gian do server cộng dồn theo `accrue`, còn passage timer nhận giây từ client. Giải thích hai cơ chế riêng, và giải thích vì sao snapshot band cần version. Nguồn: `ReadingTestPracticeController.java`, `AcademicBandService.java`, migration V8, V9, V15–V18.

7 Từ điển, ghi chú và highlight theo từng lượt làm

7.1 Chọn chữ và tra từ

Khi user chọn chữ trong passage, `ReadingPage.tsx` dùng DOM `Selection`/`Range` để tính start/end offset tương ứng body; phím T bật tra từ, H bật highlight. Khung kết quả được đặt cạnh vùng chọn. Mode REAL_TEST tắt tra từ ở UI, và `VocabularyController` cũng từ chối lookup/lưu note nếu có REAL_TEST đang IN_PROGRESS. Server không dựa duy nhất vào nút bị ẩn.

`GET /api/vocabulary/lookup` đi vào `DictionaryLookupService`. Service chuẩn hóa từ, tìm `vocabulary_entries` và `vocabulary_entry_search_terms`. Có cache hợp lệ thì trả senses ngay. Thiếu cache hoặc cần bổ sung ví dụ thì gọi Merriam-Webster Learner's Dictionary bằng Spring `RestClient`; API key

nằm ở backend. Dữ liệu JSON được parse bằng Jackson. `matchingEntries` chỉ giữ headword khớp từ tìm hoặc stem một từ phù hợp, tránh các headword không liên quan do provider gợi ý. Provider lỗi nhưng có cache thì trả cache; không có cache thì trả lỗi dịch vụ.

Một transaction lưu entry dùng chung, meaning, phonetic/audio, sense, related words và search term. `ON CONFLICT DO NOTHING` cùng UNIQUE(language_code, normalized_word) giảm trùng khi hai user tra cùng lúc. `FOR UPDATE` trên entry tuần tự hóa bổ sung senses. Khi đọc lại, service dùng một query lấy entries và một query JOIN senses/related, gom hàng bằng LinkedHashMap; không query một lần cho từng nghĩa. Audio gắn với phonetic của sense; note chọn sense nào thì sổ từ lấy phát âm/audio đúng sense đó.

Điểm phỏng vấn: Cache là dữ liệu từ điển dùng chung, không phải sổ từ từng user. Nó giảm số lần gọi API ngoài nhưng phải cân nhắc cập nhật khi nguồn đổi và tránh cache kết quả gợi ý sai. Nguồn: `DictionaryLookupService.java`, `VocabularyController.java`, V11, V12, V23.

7.2 Ghi chú cá nhân và ngữ cảnh

Ở màn từ vựng, user tra bên ngoài bài đọc và được phép để trống ngữ cảnh; frontend tự điền ví dụ của nghĩa được chọn nếu provider cung cấp. Trong passage, ví dụ từ điển vẫn hiển thị riêng nhưng `contextSentence` lưu câu lấy từ bài đọc. `readingContext.ts` dùng `Intl.Segmenter('en', {granularity:'sentence'})`, xử lý một số chữ viết tắt; nếu câu không rõ hoặc quá dài thì lấy tối đa 10 từ trước và 10 từ sau trong cùng đoạn. API giới hạn context tối đa 4000 ký tự. Đây là thuật toán frontend, không phải NLP hoàn hảo; backend hiện không đối chiếu nội dung context thật sự với body.

`POST /api/vocabulary-notes` nhận senseId, selectedText, context tùy chọn, customMeaning, passageId và danh mục có thể rỗng. Backend xác minh sense tồn tại, passage nếu có đã PUBLISHED và từng category thuộc user. `vocabulary_notes` gắn user và sense, lưu `selected_definition` như nghĩa đã chọn lúc lưu; `note_categories` biểu diễn nhiều–nhiều. Tổng từ của user lấy từ toàn bộ notes, độc lập filter danh mục. Status NEW, LEARNING, MASTERED phục vụ tiến độ sổ từ.

V20 đặt unique index cho một lần xuất hiện: user + sense + passage hoặc null + hash context. Lưu lại đúng tổ hợp sẽ trả note cũ và chỉ bổ sung danh mục mới. Một từ ở hai đoạn hoặc ngữ cảnh khác nhau vẫn có thể tạo hai note, đúng với mô hình lưu theo lần xuất hiện. Nếu customMeaning mới mâu thuẫn note cũ, API yêu cầu sửa note cũ thay vì ghi đè. PATCH note sửa từ hiển thị, context và nghĩa cá nhân; endpoint đó không đổi sense hoặc định nghĩa từ điển đã chọn.

```
Chung: vocabulary_entries → vocabulary_senses → meanings / phonetics
Riêng: users → vocabulary_notes → note_categories → categories
```

7.3 Highlight

`reading_highlights` lưu offset, selected_text và context cạnh vùng chọn, gắn `quiz_attempt_id` hoặc `reading_test_attempt_id`. API GET/POST/DELETE nhận attempt type/ID, kiểm user sở hữu attempt và passage thuộc attempt. Khi ghi, backend kiểm offset theo chuỗi UTF-16, text chọn khớp body và vùng mới không chồng highlight khác. Chỉ attempt IN_PROGRESS được sửa; xem lại SUBMITTED chỉ đọc. Làm lại tạo attempt mới nên không mang highlight cũ; xem lại lượt cũ vẫn thấy đúng. Migration V24/V25 xử lý highlight cũ thiếu scope và không gán bừa khi lịch sử mơ hồ.

Điểm phỏng vấn: Nếu lưu highlight chỉ theo `(user_id, passage_id)`, hai lượt làm khác nhau sẽ chia sẻ cùng dữ liệu. Khóa ngoại attempt diễn đạt đúng vòng đời nghiệp vụ và cho phép truy vấn review chính xác. Nguồn: `ReadingHighlightController.java`, `frontend/src/features/reading/ReadingPage.tsx`, V24, V25.

8 Dashboard, lịch sử và nhập PDF

8.1 Dashboard và lịch sử

Dashboard gộp attempt passage lẻ và full test có status SUBMITTED trong năm và bảy ngày gần nhất. `submitted_at` được đổi sang `Asia/Ho_Chi_Minh` để nhóm theo ngày; cộng số lượt và `elapsed_seconds` từng lượt. Ngày chưa nộp có 0. Một lượt mới bắt đầu nhưng chưa nộp không tạo chấm xanh trên lịch. `PracticeHistoryController` lọc passageId tùy chọn, sắp lượt nộp mới nhất, phân trang `LIMIT 21 OFFSET ...` cho page size 20 để biết còn trang sau.

Điểm cần cải thiện: lịch sử hiện gọi `PracticeResultController.result` cho từng dòng để tính đúng/tổng, tạo mẫu N+1 query. Có phân trang không tự làm query tối ưu: cần đếm số SQL mỗi trang, cân nhắc lưu score summary hoặc tính hàng loạt bằng một query, rồi chạy EXPLAIN ANALYZE với dữ liệu lớn. OFFSET sâu cũng tăng chi phí; có thể đổi sang keyset theo `(submitted_at, attempt_id)` nếu cần.

Điểm phỏng vấn: Dùng chính ví dụ lịch sử để nêu một hạn chế hiệu năng thật và hướng sửa có đo lường. Nguồn: `ReadingDashboardController.java`, `PracticeHistoryController.java`, migration V10.

8.2 PDF import và OCR

Admin tải PDF qua `POST /api/admin/pdf-imports`. Backend kiểm MIME, header `%PDF`, PDFBox đọc được, kích thước tối đa 20 MB và 1–25 trang. File lưu bằng UUID trong volume riêng; `pdf_import_jobs` lưu owner, scope PASSAGE/FULL_TEST, trạng thái, tiến độ, lỗi và thời hạn. Transaction cùng advisory lock giới hạn tối đa ba job QUEUED/RUNNING dù nhiều admin upload đồng thời; response 202 kèm Location để frontend theo dõi.

`PdfImportWorker` nhận job bằng `FOR UPDATE SKIP LOCKED`, dùng PDFTextStripper phân loại trang có text và trang cần OCR, gom tối đa ba trang/batch. Worker chạy OpenDataLoader PDF CLI; trang ít text đi qua hybrid OCR service. Có timeout, mã lỗi, cập nhật processed_pages, phục hồi RUNNING khi API khởi động lại và dọn job hết hạn. Kết quả thành block theo trang trong `review_data` JSONB, không ghi thẳng vào passages.

`PdfImportReviewService` dựng đề xuất passage/câu từ block bằng biểu thức chính quy, để lại vùng chưa phân loại và ô đáp án cần admin xác nhận. PUT review kiểm phạm vi trang, số câu, dạng câu và kích thước; `review_version` tăng bằng `WHERE version = ?` để tránh hai tab ghi đè. Hiện flow dừng ở bản rà soát có thể lưu. Schema có trạng thái DRAFT_SAVED/PUBLISHED và cột `applied_*`, nhưng chưa có endpoint áp dụng review thành passage/câu hỏi và publish tự động; admin vẫn cần biên soạn/kiểm tra qua luồng nội dung.

```
UPDATE pdf_import_jobs SET review_draft = ?::jsonb,
review_version = review_version + 1
WHERE pdf_import_job_id = ? AND review_version = ?
```

Điểm phỏng vấn: PDF parsing và OCR là khâu trích xuất, không đồng nghĩa câu hỏi/đáp án đã đáng tin để xuất bản. Review là ranh giới dữ liệu có người kiểm tra. Nguồn: `AdminPdfImportController.java`, `PdfImportService.java`, `PdfImportWorker.java`, `PdfImportReviewService.java`, V21, V22.

9 Kiến thức Java và OOP gắn với code

Khái niệm	Ví dụ trong dự án	Điều cần nói đúng
Encapsulation	private final JdbcTemplate, private helpers, DTO nội bộ	Giấu trạng thái/chi tiết; content controller hiện vẫn ôm nhiều trách nhiệm

Khái niệm	Ví dụ trong dự án	Điều cần nói đúng
Abstraction	PasswordEncoder, Principal, JdbcTemplate	Code phụ thuộc hợp đồng; BCryptPasswordEncoder là implementation được inject
Polymorphism	PasswordEncoder.matches, JdbcTemplate RowMapper callback	Dùng đa hình qua framework và lambda, chưa có class domain kế thừa lớn
Inheritance	Controller/Service không cần extends nghiệp vụ	Dự án ưu tiên composition và DI; không bịa hierarchy
Record	Credentials, Result, GroupInput, Entry, Sense	DTO gọn, bất biến theo tham chiếu; List bên trong vẫn có thể mutable
Collections/generics	List câu theo thứ tự, Set loại trùng, Map gom row	LinkedHashMap giữ insertion order; HashSet kiểm ID/candidate
Streams/lambda	flatMap câu, sum/count, RowMapper lambda	Nêu rõ phép biến đổi; stream không tự tối ưu database
Time và số	Instant, Duration, ZoneId, BigDecimal	Instant cho mốc, ZoneId cho ngày user, BigDecimal cho band 0.5
Exception	ResponseStatusException, DuplicateKeyException	400 input, 401 chưa xác thực, 404 không thấy, 409 xung đột, 422 publish thiếu

Constructor injection nghĩa là Spring tạo JdbcTemplate, PasswordEncoder hoặc service rồi truyền vào object phụ thuộc. Đây là IoC: ứng dụng khai báo dependency, framework quản lý vòng đời bean. `final` ở field chặn đổi reference sau khởi tạo; nó không làm JdbcTemplate hay List trở nên bất biến. `static final` hợp cho hằng số như tập task variant, thời hạn và regex. `String` bất biến nên chuẩn hóa tạo chuỗi mới; `StringBuilder` được dùng để lắp SQL có filter tùy chọn trong `VocabularyController.findNotes`, còn giá trị filter vẫn bind bằng `?`.

Dự án dùng `record` nhiều hơn class domain. Trong `Account` hoặc `Credentials`, accessor là `email()` thay vì `getEmail()`. `equals` của record so component; chuỗi trong Java so nội dung bằng `.equals`, không dùng `==`. `Integer`/`Long` có thể null cho dữ liệu tùy chọn như partNumber/passageId, còn `int`/`long` cho dữ liệu bắt buộc. Record không tự thay validation, quyền truy cập hoặc transaction. Các collection mutable trong record có thể bị thay từ bên ngoài nếu không copy bằng `List.copyOf`.

`AnswerTextPolicy` là lớp `final` chứa static method và private constructor vì nó không giữ state; đây là một cách gom pure utility. `AcademicBandService` là bean có dependency và hành vi truy vấn, không nên làm static. `PdfImportWorker` là component theo lịch, tương tác với process ngoài và database; việc xử lý timeout/cancellation cần ranh giới rõ hơn pure function. `PresentationTemplateCodec` tách serialization khỏi controller, còn `PresentationTemplate` record mô tả cấu trúc dữ liệu.

Điểm phỏng vấn: Nếu được hỏi bốn tính chất OOP, lấy ví dụ có thật cho đóng gói, trừu tượng và đa hình. Với kế thừa, nói dự án ưu tiên composition, không cố gán một ví dụ không tồn tại. Nêu thêm một refactor hợp lý: tách quy tắc chấm/biên soạn khỏi controller để unit test dễ hơn, nhưng tránh viết thêm abstraction khi chưa có nhu cầu thực.

10 Transaction, cạnh tranh, bảo mật và hiệu năng

10.1 Các bất biến cần bảo vệ

Tình huống	Cơ chế hiện có	Ranh giới
Hai request đăng ký cùng email	UNIQUE email_normalized; bắt DuplicateKeyException	Frontend validation không đủ
Hai request tạo attempt	Lock user row và active marker UNIQUE	Một lượt active cho user và nội dung
Heartbeat chạm submit	FOR UPDATE trên attempt; kiểm status	Request sau thấy trạng thái đã commit
Hai tab sửa PDF review	WHERE review_version = ? rồi tăng version	Request sau nhận 409, cần tải lại
Hai tab đổi password	WHERE password_hash = hash cũ	Chặn lost update cột hash; chưa thu hồi session khác
Hai user tra cùng từ	UNIQUE, ON CONFLICT, FOR UPDATE entry	Tránh trùng cache; vẫn có thể gọi provider đồng thời

Một transaction là đơn vị ACID: các thay đổi về một hành động nghiệp vụ được commit cùng nhau hoặc rollback khi lỗi. Ví dụ sửa nhóm câu hỏi cần cập nhật group, xóa/cập nhật/thêm question, viết answer rows rồi renumber; nếu nửa sau thất bại mà nửa trước đã lưu, đề sẽ hỏng. `@Transactional` bọc toàn bộ thao tác. Read-only transaction chủ yếu diễn đạt ý định và giữ cách đọc nhất quán trong một số flow, không thay việc chọn SQL hợp lý.

SQL dùng dấu `?` để bind giá trị nhằm tránh SQL injection từ input thông thường. Vài chỗ tạo SQL động cho tên cột hoặc số placeholder, nhưng tên cột được chọn từ nhánh code chứ không đưa thẳng từ request. Dữ liệu cá nhân phải có điều kiện `user_id = principal.getName()` hoặc kiểm owner trước khi trả. Quyền ADMIN là cổng vào `/api/admin/**`; một admin hợp lệ vẫn cần các ràng buộc trạng thái DRAFT/PUBLISHED. DTO learner không chứa key trước submit.

HTTP status cho phép frontend phân biệt lỗi: 400 dữ liệu không hợp lệ, 401 chưa đăng nhập/sai thông tin đăng nhập, 404 không có tài nguyên thuộc user, 409 conflict do trạng thái hoặc ghi đồng thời, 422 nội dung chưa đủ để publish. Nhiều controller dùng `ResponseStatusException` trực tiếp, chưa có một lớp domain exception/global error contract thống nhất. Khi mở rộng có thể thêm `@RestControllerAdvice` để mã lỗi và thông điệp nhất quán, nhưng không nên tự nhận hệ thống đã có nếu chưa viết.

10.2 Rủi ro trước khi chạy online

Hiện đã có BCrypt, session, CSRF, HttpOnly/SameSite, role guard, ownership check, parameterized SQL và trạng thái nội dung. Những biện pháp đó chưa đủ để kết luận hệ thống không có lỗ hổng. Công việc trước public launch gồm rate limit và audit đăng nhập, xác minh email/khôi phục mật khẩu, HTTPS và Secure cookie, secret management, backup/restore database, policy về session nhiều instance, kiểm tra XSS/CSP, giới hạn request và logging an toàn, kiểm quyền của mọi endpoint mới. Không log Credentials, API key hoặc session ID.

Với nội dung PDF và tra từ bên ngoài, network call có thể chậm/hỏng. Worker PDF đã có timeout và mã lỗi; từ điển có cache fallback. Cần quan sát tỷ lệ lỗi, thời gian đáp ứng và kích thước dữ liệu thực. Import PDF còn có giới hạn job queue, file size, page count và thời hạn lưu; đây là ví dụ xử lý resource exhaustion nhưng chưa thay một hệ queue độc lập cho tải lớn.

10.3 Query, index và N cộng một

Index phải đi theo mẫu WHERE/JOIN/ORDER BY thật. V10 tạo `(user_id, status, submitted_at)` cho dashboard/history; V11 tạo index notes theo user và created_at. Dùng EXPLAIN ANALYZE với dữ liệu đại diện để xem scan, sort, số hàng và thời gian. Thêm index tùy tiện làm tăng chi phí INSERT/UPDATE. Lịch sử có `LIMIT/OFFSET` nhưng gọi result cho từng item, tạo N+1 SQL; đây là việc nên đo và sửa đầu tiên bằng một batch query hoặc snapshot summary. `VocabularyController.findNotes` giới hạn 200 note và dùng `LIKE %term%`; khi dữ liệu tăng cần phân trang thật và có thể cân nhắc index tìm kiếm phù hợp. OFFSET sâu có thể đổi keyset theo `(submitted_at, id)`.

Điểm phỏng vấn: Có phân trang không đồng nghĩa query tối ưu. Cần xem số query bên trong mỗi trang, số cột tải về, JOIN, index và kế hoạch thực thi. Hãy nêu hạn chế thực tế trong `PracticeHistoryController` và cách đo trước khi sửa.

11 Kiểm thử và đối chiếu checklist Backend Intern

Mã nguồn có HTTP test cho auth, tải passage/full test, question group, vocabulary note và PDF import/review; `DictionaryLookupServiceTest` kiểm logic lọc kết quả từ điển. `pom.xml` có JUnit/Spring Boot Test, MockMvc và H2 ở test scope. Một số flow dùng PostgreSQL-specific SQL (`jsonb`, `ON CONFLICT`, advisory lock, `FILTER`), nên H2 không đủ chứng minh hành vi thật của PostgreSQL. Testcontainers chưa nằm trong dependency; không ghi là đã áp dụng. Khi cần kiểm chứng gần production, integration test nên khởi động PostgreSQL riêng, chạy Flyway rồi kiểm transaction, constraint, ownership, publish, submit và race condition.

Tiêu chí checklist	Đã thể hiện	Cần ôn hoặc bổ sung
Java Core và OOP	Record, collections, generics, lambda, exception, time API, DI	equals/hashCode, Big O, mutable state, thread safety
Spring MVC/Security	REST, HTTP status, session, CSRF, role, transaction	Bean Validation tập trung, tách service/repository, hardening login
PostgreSQL	PK/FK/CHECK/UNIQUE, JOIN, index, lock, Flyway	EXPLAIN ANALYZE, isolation levels, đo với dữ liệu thật
JPA/Hibernate	Chưa dùng, dự án chọn JdbcTemplate	Học entity mapping, lazy/eager, N+1, @Version qua bài tập riêng
JWT	Chưa dùng, hiện dùng session	Học claims, expiry, refresh, revoke và so sánh với session
Testing	HTTP tests, MockMvc, H2 và kiểm thử thủ công	Testcontainers/PostgreSQL, test đồng thời, end-to-end
Docker/Git/React	Compose, Dockerfile, Vite/React, Git	Release, migration rollback, CI trước chạy online

Ưu tiên kiểm thử khi bạn tự thao tác: learner không đọc/sửa attempt của người khác; published DTO không lộ key; full test không publish thiếu/dư câu; BLANK chấm đúng case/variants nhưng chặn quá số từ; nộp chỉ một lần; REAL_TEST hết giờ; note trùng cùng context không nhân đôi; highlight đúng attempt; PDF review xung đột version; cache từ điển hoạt động khi provider lỗi. Không khẳng định test suite xanh chỉ vì tồn tại file test; phải chạy lại với môi trường cụ thể và giữ bằng chứng kết quả.

12 Cách kể dự án trong phỏng vấn

12.1 Bản trình bày khoảng năm phút

1. Bối cảnh: Tôi làm ứng dụng luyện IELTS Academic Reading, có admin biên soạn passage và full test; người học làm bài, xem kết quả, lưu từ, theo dõi thống kê.
2. Kiến trúc: React/TypeScript gọi REST của Spring Boot. Spring Security dùng session và CSRF; dữ liệu ở PostgreSQL, schema do Flyway quản lý. Tôi dùng JdbcTemplate để kiểm soát SQL và nhìn rõ JOIN/transaction.
3. Luồng nội dung: Admin tạo passage DRAFT, nhóm câu theo OPTION/BLANK/MATCHING, hệ thống kiểm câu/key trước publish. Full test ghép đúng ba passage, 40 câu với số toàn đề liên tục.
4. Luồng người học: Mỗi lần làm là một attempt riêng. Backend kiểm ownership khi lưu, giữ raw/normalized answer, tính giờ, chấm sau submit; full test ghi snapshot band theo bảng quy đổi có phiên bản.
5. Dữ liệu phụ: Dictionary cache chung tránh gọi API lặp; user note theo sense và ngữ cảnh riêng. Highlight gắn attempt. PDF import tạo bản rà soát, admin kiểm trước khi đưa vào nội dung.
6. Đánh đổi: Controller còn chứa nhiều SQL, lịch sử có N+1, H2 chưa thay PostgreSQL integration test, PDF review chưa tự áp dụng thành draft. Tôi biết vị trí sửa và cách đo hiệu quả.

12.2 Câu hỏi nên tự trả lời thành tiếng

1. Record Credentials có lưu quyền trong session không? Session thật sự lưu gì và cookie chứa gì?
2. Vì sao BCrypt không thể so sánh hai hash bằng dấu bằng? Tại sao là 72 byte thay vì 72 ký tự?
3. Frontend đã kiểm email rồi, vì sao vẫn cần backend validation và UNIQUE?
4. CSRF bảo vệ request nào? Vì sao session khiến CSRF trở thành vấn đề?
5. Trong dự án này JdbcTemplate khác JPA ở mapping và kiểm soát query thế nào?
6. Vì sao cần transaction và FOR UPDATE khi submit, publish và cập nhật đồng hồ?
7. Bảo đảm 40 câu liên tục thế nào mà không mặc định passage nào luôn có 13 câu?
8. Snapshot band là dữ liệu nào? Vì sao chưa thể nói toàn bộ đề đã được snapshot?
9. Từ điển chung và ghi chú cá nhân tách nhau ở bảng nào? Khi nào lưu cùng từ hai lần là hợp lệ?
10. Có phân trang rồi vì sao vẫn có nguy cơ N+1? Đo và sửa ở đâu?
11. PDF OCR xong có tự publish chưa? Admin xác nhận ở bước nào?
12. H2 test khác PostgreSQL thật ra sao? Testcontainers cần kiểm bất biến nào?

12.3 Câu mô tả CV trung thực

- Xây dựng ứng dụng IELTS Academic Reading bằng Java 21, Spring Boot, PostgreSQL và React/TypeScript; thiết kế biên soạn passage, full test 40 câu và lịch sử lượt làm.
- Thiết kế schema bằng Flyway với khóa ngoại, ràng buộc, chỉ mục và transaction; xử lý ghi cạnh tranh bằng row lock, UNIQUE và compare-and-swap ở các luồng phù hợp.
- Tích hợp Merriam-Webster Learner's Dictionary qua RestClient; cache từ điển dùng chung và lưu ghi chú, phát âm, ngữ cảnh riêng theo user.
- Xây dựng pipeline nhập PDF có phân loại trang text/OCR, job queue và bản rà soát có version; dữ liệu cần admin kiểm tra trước khi xuất bản.

Chỉ đưa số người dùng, cải thiện hiệu năng, mức độ bảo mật hoặc tỷ lệ test pass vào CV khi có số liệu thật. Không ghi JPA, JWT, Testcontainers hay tự động publish PDF như công nghệ đã làm.

13 Bản đồ mã nguồn để tự đọc lại

Muốn ôn	Đọc theo thứ tự
Đăng nhập	frontend/src/App.tsx → frontend/src/api.ts → identity/SecurityConfiguration.java → identity/AuthController.java → migration V1
Admin nội dung	content/AdminPassageController.java → AdminQuestionGroupController.java → PresentationTemplate.java → AdminReadingTestController.java → V2–V6, V8
Làm passage	ReadingPassageController.java → PracticeAttemptController.java → AnswerTextPolicy.java → PracticeResultController.java → V7, V10, V13, V14
Làm full test	ReadingTestPracticeController.java → AcademicBandService.java → V8, V9, V15–V18
Từ vựng/highlight	frontend/src/features/reading/readingContext.ts → DictionaryLookupService.java → VocabularyController.java → ReadingHighlightController.java → V11, V12, V19, V20, V23–V25
Dashboard/lịch sử	ReadingDashboardController.java → PracticeHistoryController.java → V10
PDF	AdminPdfImportController.java → PdfImportService.java → PdfImportWorker.java → PdfImportReviewService.java → V21, V22
Kiểm thử	backend/src/test/java/vn/tuanthongma/ieltsreading, backend/pom.xml, compose.yaml

Cách ôn: tự vẽ request flow trên giấy, chỉ nơi kiểm quyền và transaction, mở schema tìm FK/UNIQUE tương ứng, rồi mô phỏng một lỗi cạnh tranh hoặc input sai. Với mỗi tính năng, trả lời ba câu: dữ liệu vào đâu, bất biến nào được giữ, và hai request đến cùng lúc thì chuyện gì xảy ra.